2025F0272

哈尔滨工业大学

大学生创新训练计划项目验收书

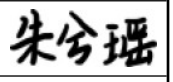
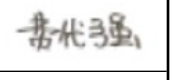
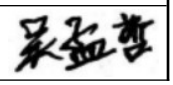
2025 10 2026 10

2024111572

18967955725 **2290549269@qq.com**

13069887146 **wangzh@hit.edu.cn**

2026 10 4

			2024	2024111572	330782200608272625	
			2024	2024111291	511323200402252111	
			2024	2024110057	130928200512100031	
			2023	2023113396	350103200508020012	

1		
2		LLM CART XGBoost
3		
4		OPD
5		

( ) *6*
( ) *6*
2.1 数据扩充技术研究背景 **6**
2.2 研究基础与主要问题 **7**
2.3 研究意义 **9**
( ) *9*
3.1 项目总技术研究路线 **9**
3.2 表格数据特征扩充 **10**
3.2.1 基于大语言模型的可解释特征生成 11
3.2.2 基于 **CART** 路径与 **XGBoost** 残差的生成引导 11
3.2.3 候选公式的执行与约束验证 12
3.2.4 验证集择优与反馈迭代 12
3.2.5 独立评价与特征贡献分析 12
3.3 数学推理链扩充 **13**
3.3.1 推理结构解析与节点遮盖 13
3.3.2 推理节点恢复与多维验证 14
3.3.3 反馈优化与推理链更新 14
3.3.4 **OPD** 模型接入与能力验证 15
3.4 代码数据扩充 **15**
3.4.1 多策略代码改写与候选生成 16
3.4.2 语义等价与代码质量验证 16
3.4.3 基于 **OPD** 的方向选择与反馈优化 16
3.4.4 模型训练与独立能力评价 17
( ) *17*
4.1 表格特征扩充与模型性能验证成果 **17**
4.1.1 代码数据扩充与模型训练成果 **18**
4.1.2 特征扩充规模与独立测试结果 **18**
4.1.3 三类表格数据的代表性特征案例 **19**
4.1.4 弱势类别改善与特征贡献验证 **23**
4.1.5 成果小结 **25**
4.2 数学推理链扩充与模型能力验证成果 **25**
4.2.1 实验设置与接收条件 25
4.2.2 反馈优化与有效扩充结果 26
4.2.3 扩充质量与分数据集表现 27
4.2.4 典型扩充与拒绝案例 29
4.2.5 **OPD** 模型能力独立对照 30
4.2.6 成果小结 31

4.3 代码数据扩充与模型训练成果 **32**
4.3.1 代码改写扩充结果 32
4.3.2 典型代码扩充案例 32
4.3.3 **OPD** 训练前后性能对比 34
4.3.4 代码生成质量分析 35
4.3.5 消融与训练过程分析 36
4.3.6 成果小结 38
4.4 ..... **38**
( ) ***38***
( ) ***38***
6.1 项目实施中的主要问题 **38**
6.2 项目体会与收获 **39**
( ) ***40***
( ) ***40***
( ) ***42***

( )

“ → → → → ” ,

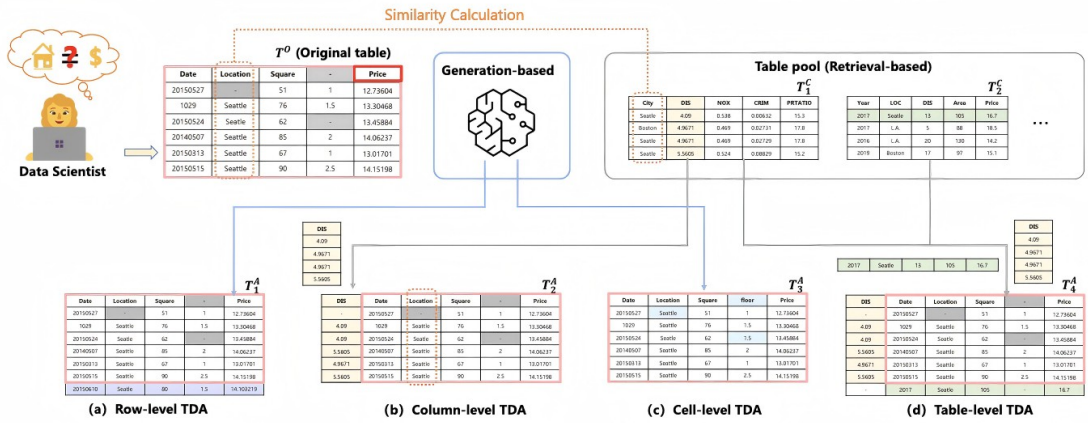
OCTree XGBoost

On-Policy Distillation OPD

( )

2.1

[1]



2.1-1 [1]

[8,9] [10]

[1,14] [28,29] [30,31]

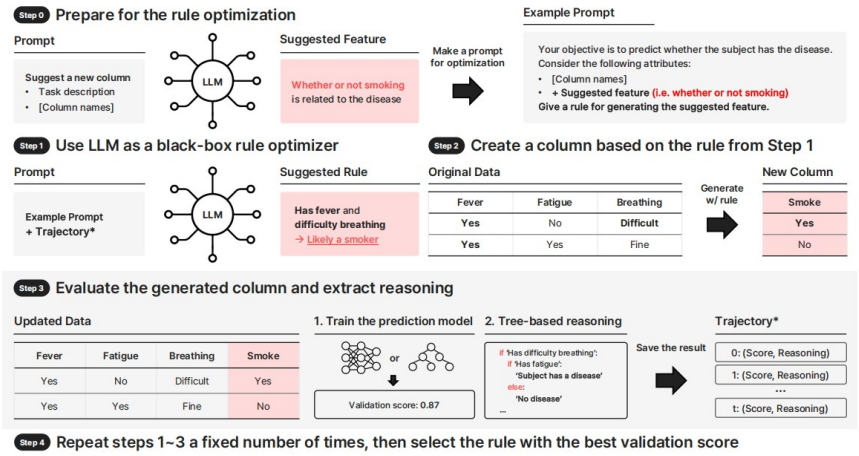
[15,32]

2.2

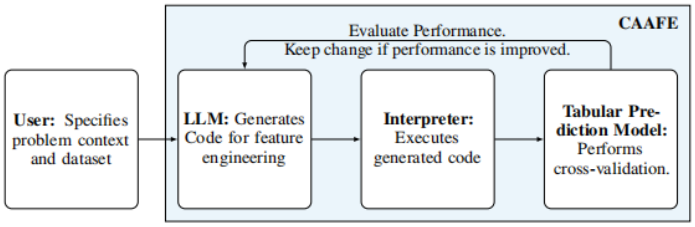
1 2.2-1 Nam[7] 2.2-2 Hollmann [15]

2.2-3 Huang [23]

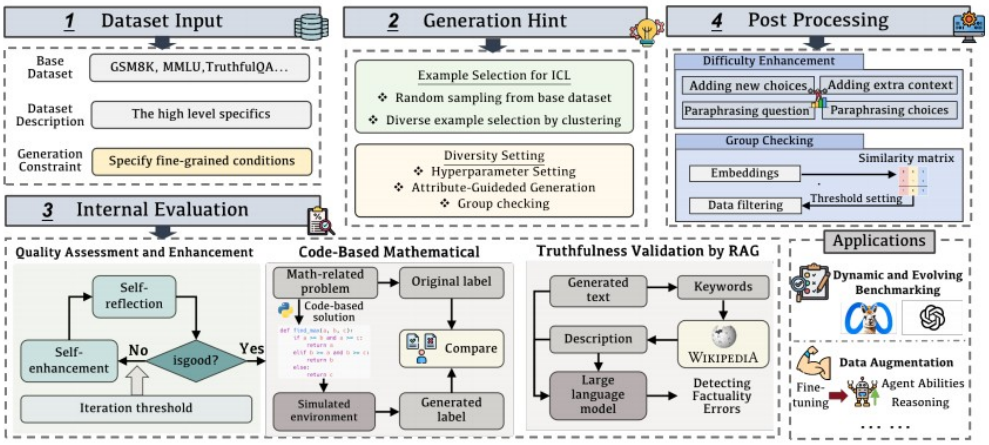
RAG [22]



2.2-1: OCTree [7]

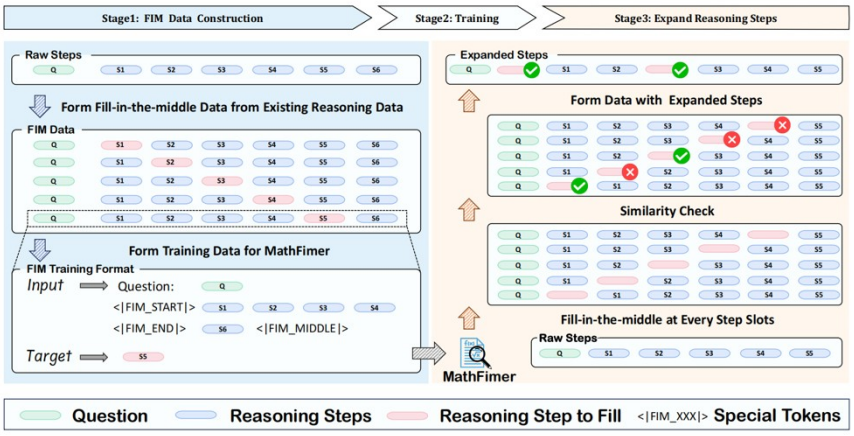


2.2-2 CAAFE [15]



2.2-3: DataGen [23]

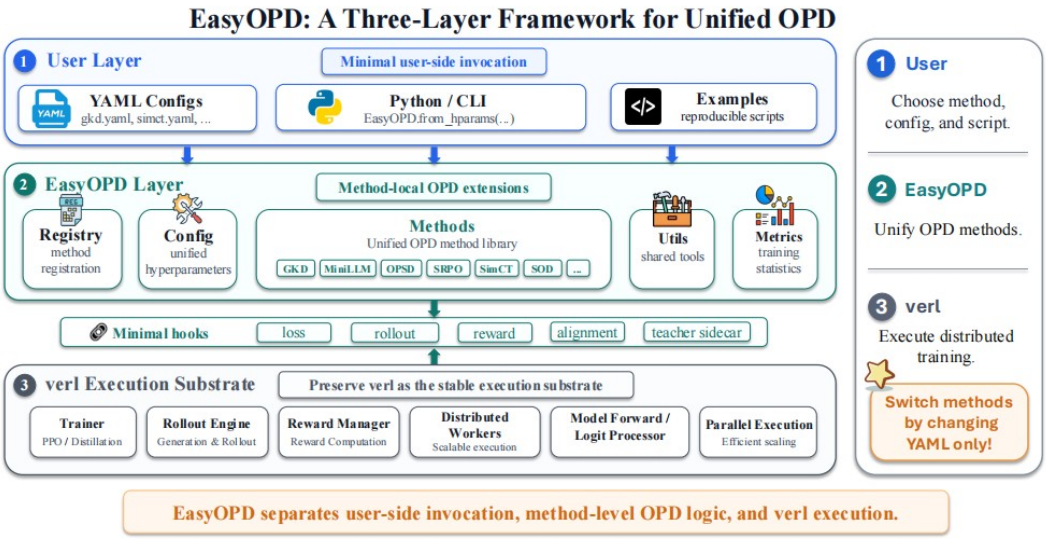
2 [27] 2.2-4 MathFimer [34]



2.2-4: MathFimer [34]

3 HumanEval [30]

[32] 2.2-5 [33,35]



2.2-5 EasyOPD [35]

“ → → → → ”

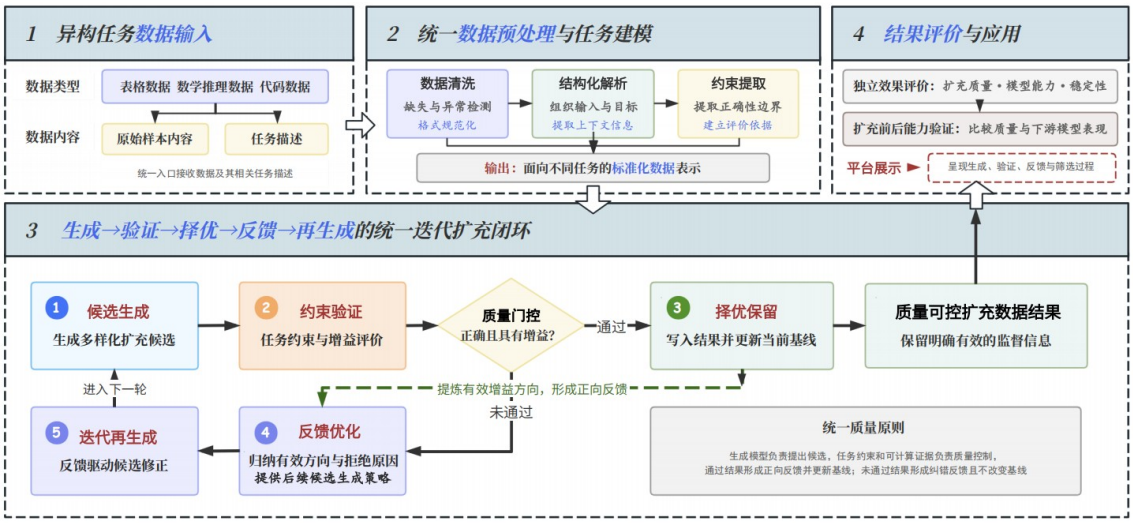
2.3

( )

3.1

3.1-1 “ → → → →

”

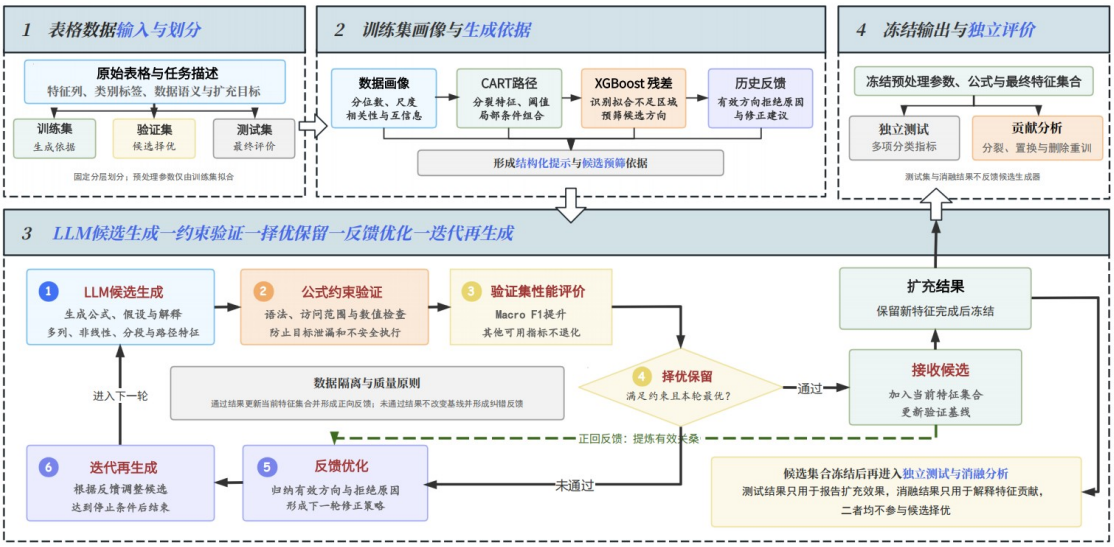


3.1-1

3.2

3.2-1 **CART**

XGBoost



3.2-1

3.2.1

LLM [15]

CART XGBoost

[15] CART

3.2.2 CART XGBoost

OCTree [7] CART

XGBoost [11]

$$r_i = e_{y_i} - p_i$$

e_{y_i} p_i XGBoost

CART

3.2.3

CART

3.2.4

XGBoost **Macro F1**

Accuracy Weighted F1 [36] Balanced Accuracy [37]

OVR Macro AUC [38] Log Loss [39]

Macro F1

→→ →→

3.2.5

XGBoost

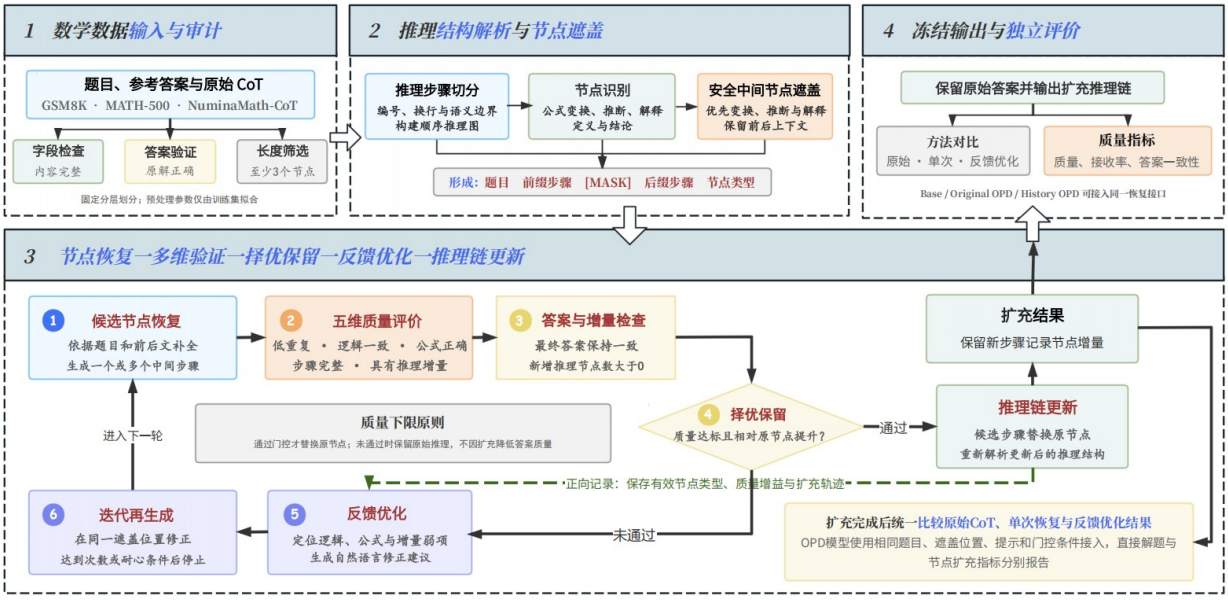
Jungle Chess Balance Scale Chess KRK

3.3

[27]

MathFimer [34] 3.3-1

GSM8K MATH-500 NuminaMath-CoT



3.3-1:

3.3.1

MathFimer [34]

3.3.2

MathFimer[34]

[28]

$$Q(c)=0.18\,S_{rep}+0.24\,S_{logic}+0.24\,S_{formula}+0.17\,S_{complete}+0.17\,S_{gain}$$

c S_{rep} S_{logic} $S_{formula}$ $S_{complete}$ S_{gain}

$$Accept(c)=I\left[Q(c)\geq\tau\wedge Q(c)-Q(o)>\delta\wedge A(c)=1\wedge\Delta N(c)>0\right],$$

o τ 0.08 δ 0.01 $A(c)=1$ $\Delta N(c)$

3.3.3

[28]

3.3.4 OPD

[33]

Base Original OPD History OPD Base Original OPD OPD History OPD

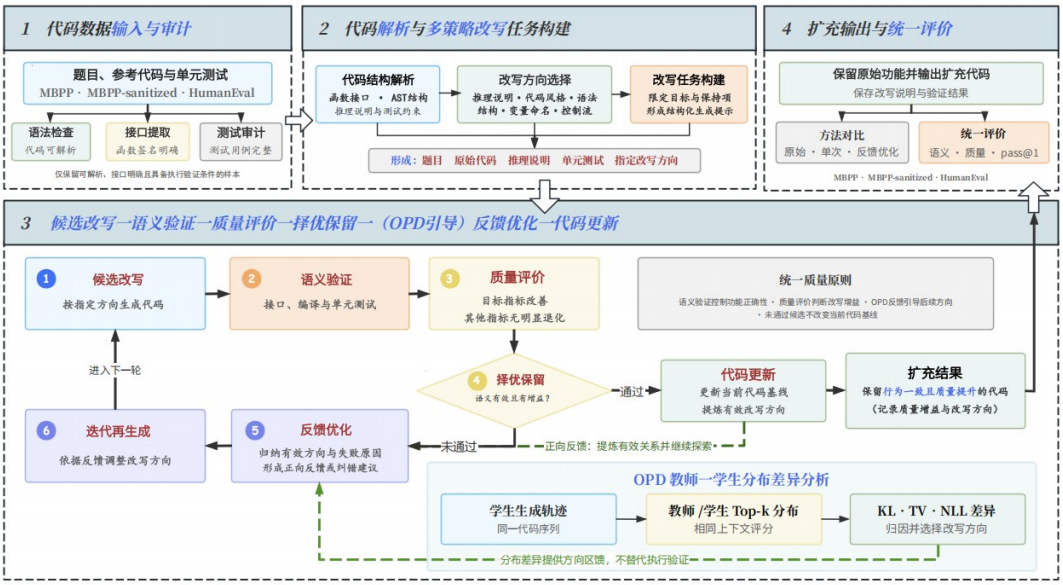
$$L_{OPD}=E_{y\sim p_S(\cdot|x)}\left[D\left(p_T(\cdot|y_{<t},x)\|p_S(\cdot|y_{<t},x)\right)\right]$$

p_S p_T

3.4

3.4-1

OPD



3.4-1:

[33] MBPP MBPP-sanitized HumanEval

[30,40]

3.4.1

3.4.2

[30]

[41]

[41–43]

3.4.3 OPD

[33] Top-k KL

$$D_{KL}\left(P_T \| P_S\right)=\sum_{v} P_T\left(v \mid x, y_{<t}\right) \log \frac{P_T\left(v \mid x, y_{<t}\right)}{P_S\left(v \mid x, y_{<t}\right)}$$

P_T P_S –

KL

a

$$R(a)=\left(D_{KL}^{(a)},\Delta NLL^{(a)},-D_{TV}^{(a)}\right),$$

KL KL

KL

→↠ →↠ →

3.4.4

MBPP OPD –

MBPP MBPP-sanitized HumanEval pass@1

( )

4.1

LLM CART

XGBoost

4.1.1

4.1-1 20% 40% 40% 42

Jungle Chess	44,819	3	**6**	8,963	17,928	17,928
Balance Scale	625	3	**4**	125	250	250
Chess KRK	28,056	18	**6**	5,611	11,222	11,223

4.1-1:

XGBoost **Macro F1** 5 5

1

4.1.2

75 LLM 8 4.1-2

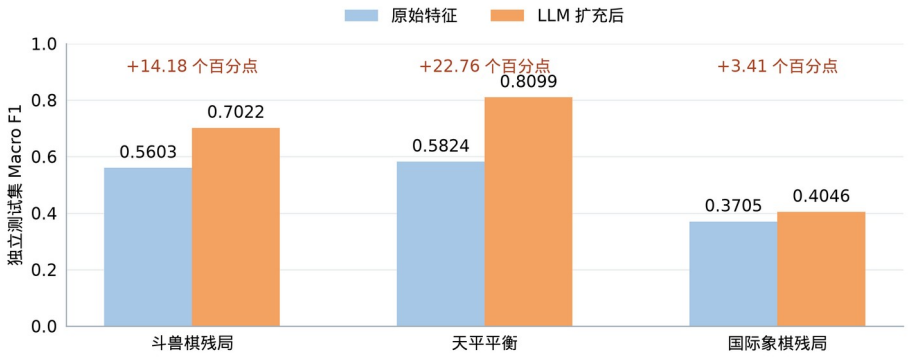
Jungle Chess	25	3	**6 → 9**
Balance Scale	25	2	**4 → 6**
Chess KRK	25	3	**6 → 9**
	75	8	**—**

4.1-2: LLM

XGBoost 4.1-3

		LLM	
Jungle Chess	0.5603	**0.7022**	**+14.18**
Balance Scale	0.5824	**0.8099**	**+22.76**
Chess KRK	0.3705	**0.4046**	**+3.41**

4.1-3: Macro F1



4.1-1: Macro F1

Macro F1 13.45

4.1.3

1 Jungle Chess

Jungle Chess

$$J_1 = sign(\Delta s)\ln(1+|\Delta s|),$$

Δs

7 6

$$\Delta s = 7-6 = 1, J_1 = \ln 2 \approx 0.6931.$$

4.1-2 3 6 9

b d

扩充前：6 列原始特征

特征名称	数值
白方棋子强度	7
白方棋子横坐标	1
白方棋子纵坐标	0
黑方棋子强度	6
黑方棋子横坐标	0
黑方棋子纵坐标	2

基线预测：b（错误）

新增特征列

扩充后：9 列输入特征

特征名称	数值
白方棋子强度	7
白方棋子横坐标	1
白方棋子纵坐标	0
黑方棋子强度	6
黑方棋子横坐标	0
黑方棋子纵坐标	2
J1 有符号对数强度差	0.6931
J2 强度与纵向位置交互	-0.0667
J3 几何与已有特征组合	-0.2650

扩充后预测：d（正确）

测试集第 12 行 | 真实类别：d | 原始数据与标签保持一致，橙色标记是生成的新列

4.1-2 Jungle Chess

J_1

2 Balance Scale

Balance Scale w_L d_L w_R d_R

$$B_1 = \tanh\left[\frac{|d_L - d_R|/5 + |w_L - w_R|/4}{2}\right].$$

CART

$$B_2=\begin{cases}\tanh\left(\left|B_1-m_{B_1}\right|/s_{B_1}\right), & d_R\le 4.5,\\ \tanh\left(\left|d_L-3\right|/2\right), & d_R>4.5.\end{cases}$$

m_{B_1} s_{B_1} B_1 4.5 CART 4 5

4.1-3 1 4 $B_1=0$ $B_2\approx 0.7616$ 4 6 L B

4.1-3 Balance Scale

B_1

3 Chess KRK

Chess KRK K_1

$$K_1=\cos\left[\frac{\pi\left(f_R-f_B\right)}{8}\right]\tanh\left[\frac{\left(f_R-5\right)+\left(f_B-5\right)}{8}\right],$$

f_R f_B

6 9 1

$$K_1=\cos(0)\tanh(-1)\approx -0.7616.$$

4.1-4 9 10

4.1-4 Chess KRK

Jungle Chess Balance Scale Chess KRK

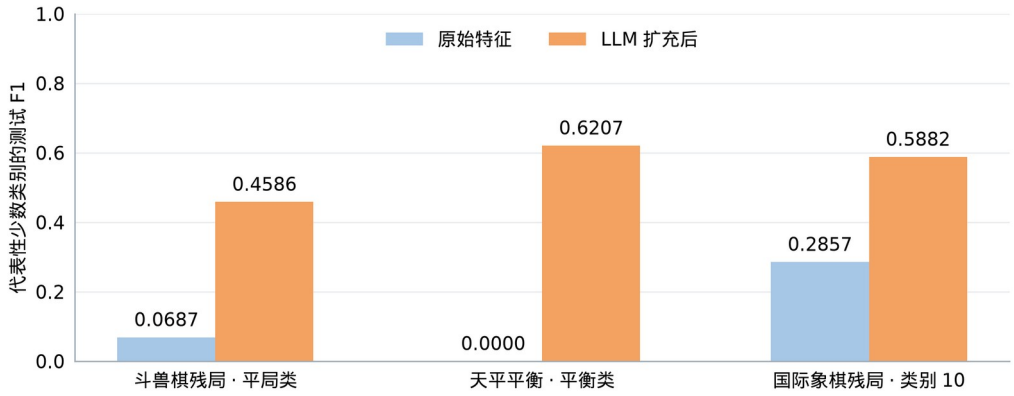
4.1.4

1

Macro F1 F1 4.1-4

Jungle Chess	0.0687	**0.4586**
Balance Scale	0	**0.6207**
Chess KRK 10	0.2857	**0.5882**

4.1-4 F1



4.1-5 F1

Balance Scale Chess KRK 10 20

11 F1

2

XGBoost

		Macro F1		
Jungle Chess J_1	244	**14.24±0.31**	**4.84**	
Balance Scale B_1	172	**21.33±2.11**	**22.76**	
Chess KRK K_1	499	**4.61±0.47**	**2.30**	

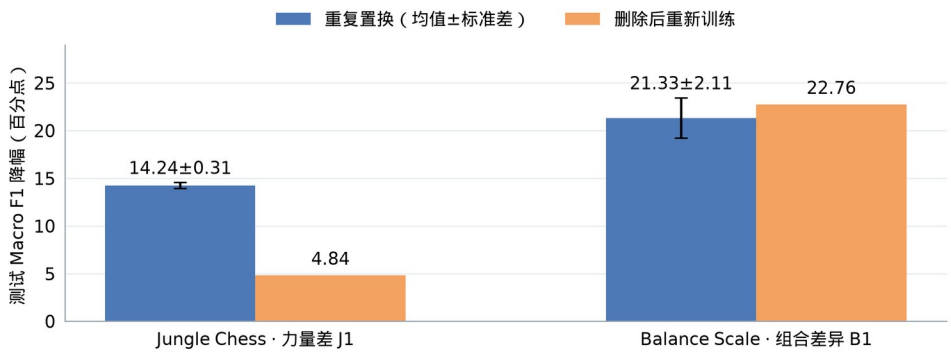
4.1-5

Macro F1 8

5

B_2 B_1 B_1 B_2

B_1



4.1-6

4.1.5

LLM 75 8

Macro F1 14.18 22.76 3.41 8

XGBoost 5

4.2

OPD

OPD

4.2.1

GSM8K MATH-500 NuminaMath-CoT 30 90

API

gpt-5.6-terra 0.2 20260927

0.80 0.01 0

4.2.2

4.2-1

			0	**0.794**
	6/90	6.7%	6	**0.798**
	19/90	21.1%	19	**0.810**

4.2-1

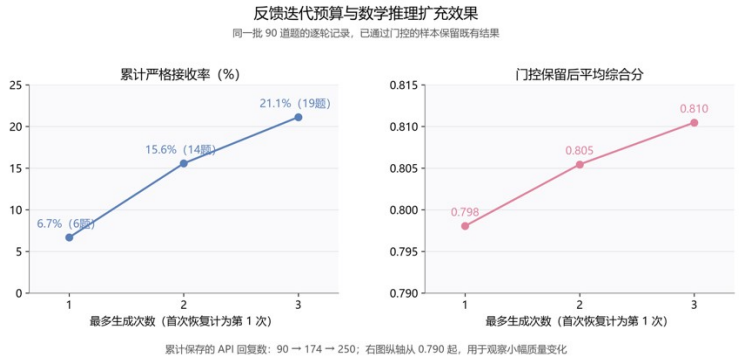
6 19 6.7% 21.1% 71 88 19

1 2 3 4.2-2

				API
1	6	6.7%	0.798	90
2	14	15.6%	0.805	174
3	19	21.1%	0.810	250

4.2-2

6 14 19 0.798 0.810 8 5



4.2-1

4.2.3

0.761

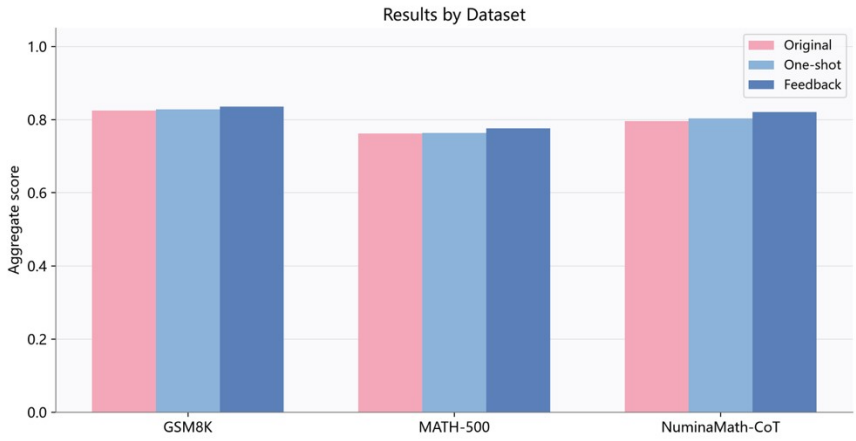
0.804 0.798 0.810 4.2-3

GSM8K	30	0.824	0.827	**0.835**	**7 23.3%**

MATH-500	30	0.761	0.764	**0.776**	**3 10.0%**
NuminaMath-CoT	30	0.795	0.803	**0.821**	**9 30.0%**

4.2-3

NuminaMath-CoT MATH-500



4.2-2

4.2-4

	0.796	0.829	**0.864**
	0.860	0.842	**0.819**
	0.644	0.651	**0.685**
	0.885	0.887	**0.892**
	0.816	0.821	**0.838**

4.2-4

0.860 0.819

100%

4.2.4

1

GSM8K gsm8k_test_59 Kim “Kim 39−8 31 ”

	Kim 39−8 31
	Shelley Kim 8 Kim 39−8
	39−8 31 Kim 31
	0.864 0.056 31 1

4.2-5

gsm8k_test_16 “12/20×100% 60%” “12/20 3/5” “3/5×100% 60%”
0.928 0.072

2

gsm8k_test_97
0.767 0.057

4.2.5 OPD

Qwen3-1.7B LoRA
Base Original OPD History OPD History OPD

siyanzhao/Openthoughts_math_30k_opsd 80 30 40
MATH

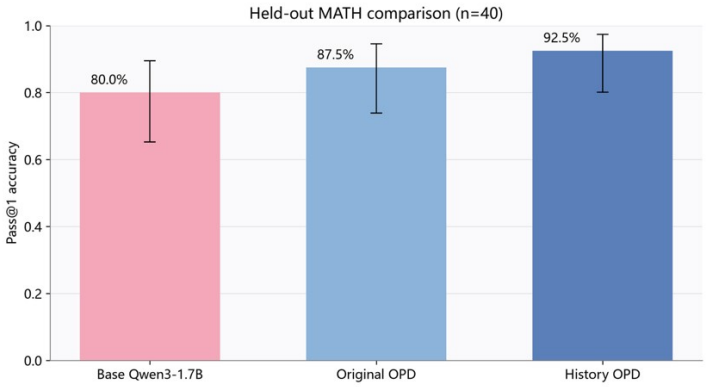
Base Qwen3-1.7B	32/40	80.0%	
Original OPD	35/40	87.5%	**7.5**
History OPD	37/40	92.5%	**12.5**

4.2-6 MATH

Original OPD History OPD 3 5 40 McNemar 0.05

OPD

OPD



4.2-4 Base Original OPD History OPD

4.2.6

90 6 19 6.7% 21.1%

100% OPD 80.0% 87.5% 92.5%

4.3

OPD

4.3.1

MBPP 100

100	884	31	69	**31.0%**

4.3.1:

4.3.1 100 884 31 69

4.3.2

Qwen3-1.7B Qwen3-4B Top-k

4.3.2

MBPP Task 601		mcl max chain_lengths max_length	3/3		
MBPP Task 603		pop			
MBPP Task 609		x max_value			
MBPP Task 602					

4.3.2:

MBPP Task 601

```
class Pair(object):
    def __init__(self, a, b):
        self.a = a
        self.b = b

def max_chain_length(arr, n):
    max = 0
    mcl = [1 for i in range(n)]
    for i in range(1, n):
        for j in range(0, i):
            if (arr[i].a > arr[j].b and
                    mcl[i] < mcl[j] + 1):
                mcl[i] = mcl[j] + 1
    for i in range(n):
        if max < mcl[i]:
            max = mcl[i]
    return max
```

扩充

```
class Pair(object):
    def __init__(self, a, b):
        self.a = a
        self.b = b

def max_chain_length(arr, n):
    max_length = 0
    chain_lengths = [1] * n
    for i in range(1, n):
        for j in range(i):
            if (arr[i].a > arr[j].b
                    and chain_lengths[i] < chain_lengths[j] + 1):
                chain_lengths[i] = chain_lengths[j] + 1
    max_length = max(chain_lengths)
    return max_length
```

4.3.1: Task 601

4.3.1 mcl max chain_lengths max_length

	26	0 ↓26
	55.5	58.2 ↑2.7
	7	5 ↓2
	100%	100%

4.3.3: Task 601

4.3.3 **Task 602**

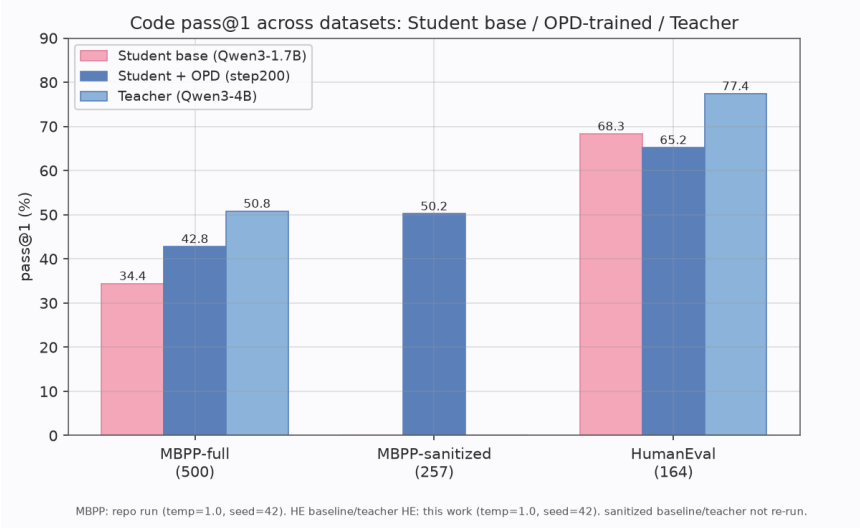
4.3.3 OPD

Qwen3-1.7B Qwen3-4B MBPP 200 OPD

4.3.4

	MBPP-full 500	MBPP-sanitized 257	HumanEval 164
	34.4%	–	68.3%
OPD step200	**42.8%↑8.4%**	**50.2%↑15.8%**	**65.2%↓3.1%**
	50.8%	–	77.4%

4.3.4: OPD



4.3.2: pass@1

MBPP-full OPD pass@1 34.4% 42.8% 8.4 84.3% OPD 35.4% 41.0% OPD MBPP-sanitized 50.2% pass@1

HumanEval OPD pass@1 68.3% 65.2% MBPP

4.3.4

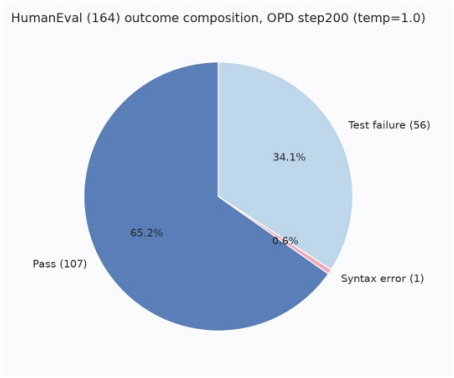
OPD 4.3.5

	MBPP-full	**MBPP-sanitized**	**HumanEval**
pass@1	41.0%	50.2%	65.2%
	100%	100%	100%
	99.0%	99.2%	99.4%
	97.4%	95.7%	98.8%
	212.5	194.0	182.5
	0.6%	0.4%	0%

4.3.5: OPD

100% 99% 95% 1024 tokens

4.3.3 HumanEval 164 107 56 1



4.3.3: HumanEval

4.3.5

100

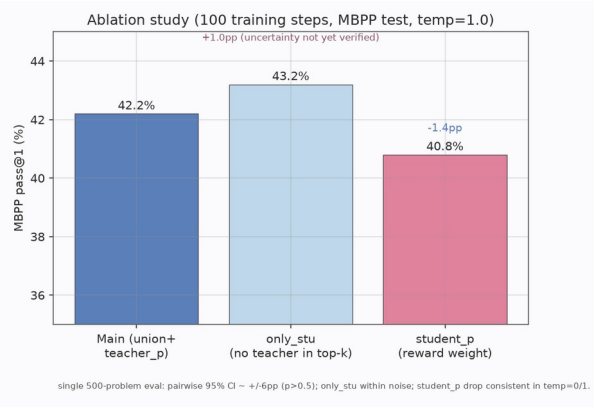
	MBPP pass@1	

Top-k	42.2%	—
Top-k	43.2%	**+1.0**
	40.8%	**−1.4**

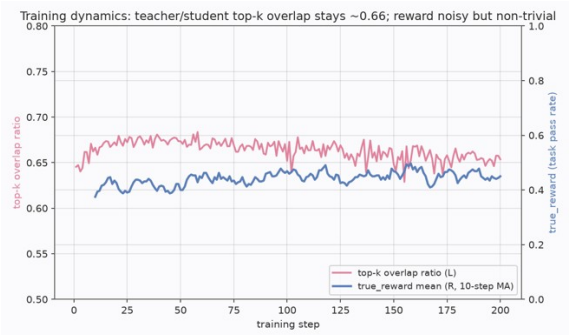
4.3.6: OPD

pass@1 1.4 Top-k 1.0

4.3.5 Top-k 0.66 0.4 0.5



4.3.4 OPD Top-k



4.3.5 Top-k

4.3.6

100 MBPP 884 31 OPD MBPP pass@1 34.4% 42.8% 99% 95%

4.4

4.4-1

( )

		460
	API token	3160
		380
		4000

( )

6.1

1

2

3

6.2

( )

1.

2.

3.

4.

5.

( )

( )

[1] Cui, L., Li, H., Chen, K., Shou, L. & Chen, G. Tabular Data Augmentation for Machine Learning: Progress and Prospects of Embracing Generative AI. Preprint at https://doi.org/10.48550/arXiv.2407.21523 (2024).

[2] Katz, G., Shin, E. C. R. & Song, D. ExploreKit: Automatic Feature Generation and Selection. in *2016 IEEE 16th International Conference on Data Mining (ICDM)* 979–984 (IEEE, Barcelona, Spain, 2016). doi:10.1109/ICDM.2016.0123.

[3] Lin, Y., Ding, B., Jagadish, H. V. & Zhou, J. SMARTFEAT: Efficient Feature Construction through Feature-Level Foundation Model Interactions. (2024).

[4] Xiao, M. *et al.* Beyond Discrete Selection: Continuous Embedding Space Optimization for Generative Feature Selection. in *2023 IEEE International Conference on Data Mining (ICDM)* 688–697 (IEEE, Shanghai, China, 2023). doi:10.1109/ICDM58522.2023.00078.

[5] Zheng, S. & Charoenphakdee, N. Diffusion models for missing value imputation in tabular data. Preprint at https://doi.org/10.48550/arXiv.2210.17128 (2023).

[6] Yoon, J., Jordon, J. & Schaar, M. GAIN: Missing Data Imputation using Generative Adversarial Nets. *ArXiv* https://www.semanticscholar.org/paper/a89f0a78f86077864e108a1bd2c4e670c85907f8 (2018).

[7] Nam, J. *et al.* Optimized Feature Generation for Tabular Data via LLMs with Decision Tree Reasoning.

[8] ImageNet classification with deep convolutional neural networks | Communications of the ACM. https://dl.acm.org/doi/10.1145/3065386.

[9] Szegedy, C. *et al.* Going Deeper with Convolutions. Preprint at https://doi.org/10.48550/arXiv.1409.4842 (2014).

[10] Devlin, J., Chang, M.-W., Lee, K. & Toutanova, K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. Preprint at https://doi.org/10.48550/arXiv.1810.04805 (2019).

[11] Grinsztajn, L., Oyallon, E. & Varoquaux, G. Why do tree-based models still outperform deep learning on tabular data? Preprint at https://doi.org/10.48550/arXiv.2207.08815 (2022).

[12] McElfresh, D. *et al.* When Do Neural Nets Outperform Boosted Trees on Tabular Data? Preprint at https://doi.org/10.48550/arXiv.2305.02997 (2024).

[13] Cherepanova, V. *et al.* A Performance-Driven Benchmark for Feature Selection in Tabular Deep Learning. Preprint at https://doi.org/10.48550/arXiv.2311.05877 (2023).

[14] Zhang, T. *et al.* OpenFE: Automated Feature Generation with Expert-level Performance. Preprint at https://doi.org/10.48550/arXiv.2211.12507 (2023).

[15] Hollmann, N., Müller, S. & Hutter, F. Large Language Models for Automated Data Science: Introducing CAAFE for Context-Aware Automated Feature Engineering. Preprint at https://doi.org/10.48550/arXiv.2305.03403 (2023).
[16] Amballa, A., Akkinapalli, G., Madine, M., Yarrabolu, N. P. P. & Grabowicz, P. A. Automated Model Selection for Tabular Data. Preprint at https://doi.org/10.48550/arXiv.2401.00961 (2024).
[17] Horn, F., Pack, R. & Rieger, M. The autofeat Python Library for Automated Feature Engineering and Selection. Preprint at https://doi.org/10.48550/arXiv.1901.07329 (2020).
[18] Dinh, T. *et al.* LIFT: Language-Interfaced Fine-Tuning for Non-Language Machine Learning Tasks. Preprint at https://doi.org/10.48550/arXiv.2206.06565 (2022).
[19] Hegselmann, S. *et al.* TabLLM: Few-shot Classification of Tabular Data with Large Language Models. Preprint at https://doi.org/10.48550/arXiv.2210.10723 (2023).
[20] Nam, J. *et al.* Tabular Transfer Learning via Prompting LLMs. Preprint at https://doi.org/10.48550/arXiv.2408.11063 (2024).
[21] Yan, J. *et al.* Making Pre-trained Language Models Great on Tabular Prediction. Preprint at https://doi.org/10.48550/arXiv.2403.01841 (2024).
[22] Ren, J. et al. Self-evaluation improves selective generation in large language models. (2023).
[23] Huang, Y. et al. DataGen: unified synthetic dataset generation via large language models. (2024).
[24] Yang, C. *et al.* Large Language Models as Optimizers. Preprint at https://doi.org/10.48550/arXiv.2309.03409 (2024).
[25] Loh, W. Classification and regression trees. *WIREs Data Min & Knowl* **1**, 14–23 (2011).
[26] Frank, A. & Asuncion, A. {UCI} Machine Learning Repository. undefined-undefined (2010).
[27] Loubna, B. A. et al. SmolLM2: When Smol Goes Big — Data-Centric Training of a Small Language Model. Preprint at https://hf.co/collections/HuggingFaceTB/smollm2-6723884218bcda64b34d7db9 (2025).
[28] LIGHTMAN H, KOSARAJU V, BURDA Y, et al. Let's Verify Step by Step[EB/OL]. 2023.
[29] GAO L, MADAAN A, ZHOU S, et al. PAL: Program-Aided Language Models[C]//Proceedings of the 40th International Conference on Machine Learning. 2023: 10764-10799.
[30] CHEN M, TWOREK J, JUN H, et al. Evaluating Large Language Models Trained on Code[EB/OL]. 2021.

[31] LE H, WANG Y, GOTMARE A D, et al. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning[C]//Advances in Neural Information Processing Systems. 2022, 35: 21314-21328.
[32] MADAAN A, TANDON N, GUPTA P, et al. Self-Refine: Iterative Refinement with Self-Feedback[C]//Advances in Neural Information Processing Systems. 2023, 36.
[33] AGARWAL R, VIEILLARD N, ZHOU Y, et al. On-Policy Distillation of Language Models: Learning from Self-Generated Mistakes[C]//International Conference on Learning Representations. 2024.
[34] YAN Y, SHEN Y, LIU Y, et al. MathFimer: Enhancing Mathematical Reasoning by Expanding Reasoning Steps through Fill-in-the-Middle Task[C]//International Conference on Learning Representations. 2026.
[35] Sun J, Zheng M, Song M, et al. EasyOPD: An Easy-to-use On-Policy Distillation Framework for Large Language Models. 2026.
[36] Grandini, M., Bagli, E. & Visani, G. Metrics for Multi-Class Classification: an Overview. *arXiv preprint arXiv:2008.05756* (2020).
[37] Brodersen, K. H., Ong, C. S., Stephan, K. E. & Buhmann, J. M. The Balanced Accuracy and Its Posterior Distribution. In *Proceedings of the 20th International Conference on Pattern Recognition*, 3121–3124 (2010). DOI: 10.1109/ICPR.2010.764.
[38] Fawcett, T. An Introduction to ROC Analysis. *Pattern Recognition Letters*, 27(8), 861–874 (2006). DOI: 10.1016/j.patrec.2005.10.010.
[39] Gneiting, T. & Raftery, A. E. Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, 102(477), 359–378 (2007). DOI: 10.1198/016214506000001437.
[40] Austin, J., Odena, A., Nye, M., et al. Program Synthesis with Large Language Models[J/OL]. arXiv preprint arXiv:2108.07732, 2021.
[41] Ren, S., Guo, D., Lu, S., et al. CodeBLEU: A Method for Automatic Evaluation of Code Synthesis[J/OL]. arXiv preprint arXiv:2009.10297, 2020.
[42] McCabe, T. J. A Complexity Measure[J]. IEEE Transactions on Software Engineering, 1976, SE-2(4): 308–320.
[43] Oman, P., Hagemeister, J. Metrics for Assessing a Software System's Maintainability[C]//Proceedings of the Conference on Software Maintenance. Orlando, FL, USA: IEEE, 1992: 337–344.

GitHub OPD code

- OPD
- **GitHub** https://github.com/zhu060827/OPD
- code
- code README.md FILE_GUIDE.md

□□ □□□□

1		OPD code
2		LLM CART XGBoost tabular_data
3		Math math math_adapter.py
4		OPD code_rewrite_feedback_expander OPD-main
5		app.py index.html